

Python Practice Problem Statements

1. Student Information & Percentage Calculator

Create a Python program that takes a student's name, roll number, and marks in 5 subjects as input.

Requirements:

- Store all values using variables.
- Calculate:
 - Total marks
 - Average marks
 - Percentage
- Display all details properly.

Additional Conditions:

- If percentage is above 90 → print "Excellent"
- If percentage is between 75 and 90 → print "Very Good"
- Otherwise → print "Needs Improvement"

Concepts Used:

- Variables
 - Data Types
 - User Input
 - Arithmetic Operators
 - if-else Conditions
-

2. Simple ATM Machine Simulation

Create a Python program that simulates a mini ATM system.

Requirements:

- Store a default PIN and account balance.
- Ask the user to enter the PIN.
- If the PIN is correct:
 - Show options:
 1. Check Balance
 2. Deposit Money
 3. Withdraw Money

- Perform the selected operation.
- If withdrawal amount is greater than balance, display "Insufficient Balance".

Additional Conditions:

- Allow only 3 incorrect PIN attempts.
- After 3 wrong attempts display "Account Locked".

Concepts Used:

- if-else
 - Nested Conditions
 - Arithmetic Operators
 - User Input
 - while Loop
-

3. Multiplication Table Generator

Write a Python program that generates multiplication tables.

Requirements:

- Ask the user to enter a number.
- Ask how many multiples should be printed.
- Use a for loop to print the multiplication table.

Example:

If user enters 5 and limit 10

Output:

5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50

Bonus Challenge:

- Print tables from 1 to the entered number.

Concepts Used:

- for Loop
 - Variables
 - Arithmetic Operations
-

4. Number Pattern Printing Program

Create a Python program to print number patterns using loops.

Requirements:

Print the following patterns:

Pattern 1

```
1
12
123
1234
12345
```

Pattern 2

```
54321
5432
543
54
5
```

Bonus Challenge:

Create your own custom pattern.

Concepts Used:

- Nested Loops
 - for Loop
 - Logic Building
-

5. Odd and Even Number Analyzer

Create a program that checks numbers between a given range.

Requirements:

- Ask the user to enter:
 - Starting number
 - Ending number
- Print:
 - All even numbers
 - All odd numbers
- Count how many even and odd numbers are present.

Bonus Challenge:

- Calculate the sum of all even numbers.

Concepts Used:

- for Loop
 - if-else
 - Arithmetic Operators
-

6. Username & Password Login System

Develop a mini login system.

Requirements:

- Store predefined username and password.
- Ask the user to login.
- If credentials are correct:
 - Print "Login Successful"
- Otherwise:
 - Print "Invalid Username or Password"

Additional Features:

- Maximum 3 login attempts.
- Display remaining attempts.
- If username is "admin":
 - Display special welcome message.

Bonus Challenge:

- Add password length validation.

Concepts Used:

- if-else
 - Nested Conditions
 - Logical Operators
 - while Loop
-

7. Electricity Bill Calculator

Create a Python program to calculate electricity bills.

Billing Rules:

- First 100 units → ₹5 per unit
- Next 100 units → ₹7 per unit
- Above 200 units → ₹10 per unit

Requirements:

- Take customer name and units consumed as input.
- Calculate total bill.
- Display the final amount.

Additional Conditions:

- Add 18% GST if bill exceeds ₹2000.
- Display "High Power Consumption" warning if units exceed 300.

Concepts Used:

- Variables
 - Arithmetic Operators
 - if-else
 - Nested Conditions
-

8. Function-Based Calculator

Create a calculator using functions.

Requirements:

Create separate functions for:

- Addition
- Subtraction
- Multiplication
- Division

Program Flow:

- Ask the user for two numbers.
- Ask for the operation choice.
- Call the appropriate function.

Additional Conditions:

- Handle division by zero.

Bonus Challenge:

- Add power and modulus operations.

Concepts Used:

- Functions
 - Parameters
 - if-else
 - Operators
-

9. Password Strength Checker

Create a Python program to check password strength.

Rules:

A password is considered strong if:

- Length is at least 8 characters
- Contains at least:
 - 1 uppercase letter
 - 1 lowercase letter
 - 1 number

Requirements:

- Ask the user to enter a password.
- Display:
 - "Weak Password"
 - "Medium Password"
 - "Strong Password"

Bonus Challenge:

- Allow repeated attempts until strong password is entered.

Concepts Used:

- Loops
 - Conditions
 - Logical Operators
 - String Checking
-

10. Quiz Game Application

Create a small quiz game in Python.

Requirements:

- Ask at least 5 questions.
- Each correct answer gives 1 point.
- Display final score at the end.

Additional Conditions:

- Use functions for asking questions.
- Display:
 - "Excellent" for score above 4
 - "Good" for score between 3 and 4
 - "Needs Practice" otherwise

Bonus Challenge:

- Add negative marking for wrong answers.

Concepts Used:

- Functions
- Loops
- Conditions
- Variables
- Logic Building